The Barocq programming language

User manual

Table of Contents

Preamble . . . 1
Installation guide . . . 2
  Using opam . . . 2
  Using make . . . 2
  Editor support . . . 2
Syntax . . . 3
  Types . . . 3
  Expressions . . . 5
  Definitions . . . 7
  Foreign function interface . . . 8
  Modules . . . 10
Compilation directives . . . 11
  More details about `@unique` . . . 11
A small example . . . 13
  A toy microkernel . . . 13
  Generating Rocq files . . . 15
Coding good practices . . . 16

Preamble

Barocq is a minimal, first-order, purely functional programming language with built-in enums, records and arrays.

The purpose of Barocq is to integrate C-like features into a functionnal language to easily write system code (e.g. microkernel). Barocq is compiled to CompCert C, the source language of the CompCert C compiler. A Barocq program can also be translated to a shallow and deep embedding in Coq/Rocq.

Even though Barocq has a purely functional semantics, the compiler transforms all record and array modifications into in-place updates in C (i.e. no copies are performed). Of course, only a very small subset of functional programs can be translated to imperative ones using in-place updates without breaking the source program's semantics. Thus, Barocq is equipped with a static analyzer that rejects programs whose compilation would be incorrect.

Installation guide

The Barocq compiler depends on OCaml, Rocq and a customized version of the CompCert compiler. First, install the OCaml Package Manager (OPAM). It is recommended to create a new opam switch with OCaml 4.14.2 for building Barocq. Once the switch is initialized, add the Rocq package repository:

```
opam repo add rocq-released https://rocq-prover.org/opam/released
```

Using opam

```
opam install --deps-only -b .
opam install .
```

Using make

```
opam install --deps-only -b .
make
make install
```

`make install` installs the binary executable `barocq` under the `bin/` folder of the current opam switch, and the Barocq Rocq files in the current opam switch.

Editor support

There is a syntax-highlighting support for VSCode. Open the VSCode command palette, look for the command `Developer: Install Extension from Location…` and choose the directory `misc/vsbarocq`.

Syntax

Types

Integers and Booleans

Barocq supports the usual boolean and integer types.

Type	Description
`bool`	boolean
`i32`	32-bit integer
`u32`	32-bit unsigned integer
`i64`	64-bit integer
`u64`	64-bit unsigned integer

Enumerated types

Enumerated types can be created with the keyword `enum`. An example an enum type definition:

```
enum retcode {
  SUCCESS,
  ERROR,
}
```

An enum constructor (like SUCCESS or ERROR) must begin with an uppercase letter.

Record types

Record types can be created with the keyword `record`. An example of two record type definitions:

```
record point {
  x: i32,
  y: i32,
}

record circle {
  center: point,
  radius: u32,
}
```

Array types

For any type `t`, an array of type `t` is written `[t]`.

NOTE	Although arrays can be of any type, Barocq does not support parametric polymorphism.

Arrays can be accessed via unsigned integers (32- or 64-bit).

Type aliases

The following code creates a type named `int` to be an alias for `i32`, and a type named `buffer` to be an alias of `[u32]`.

```
type int = i32

type buffer = [u32]
```

Type aliases is a front-end feature only, i.e. they will not appear in the compiled C file.

Unboxed values

Integers, Booleans and Enums are primitive types. By default, all **non-primitive** types are boxed in the generated C file, i.e. they are wrapped in a pointer.

For example, the following record definition:

```
record draw {
  pts: [point],
  color: u64,
}
```

is compiled to

```
struct draw {
  struct point **pts;
  unsigned long long color;
}
```

In fact, the field `pts` of record `draw` becomes a pointer to an array that itself contains pointers to values of type `point`. Unboxed values are introduced by the operator `#`. For example,

```
record draw {
  pts: [#point],
  color: u64,
}
```

is compiled to

```
struct draw {
  struct point *pts;
  unsigned long long color;
}
```

because the field `pts` is now just a pointer to an array of (unboxed) points. We can even go further by writing

```
record draw {
  pts: #[#point; 3ul],
  color: u64,
}
```

which becomes

```
struct draw {
  struct point pts[3];
  unsigned long long color;
}
```

In this last example, we also unboxed the array of points inside the record definition `draw`. Note that the size of an unboxed array must be specified to be compilable to C. The syntax is `#[t; size]` where `t` is any type. Barocq supports both unboxing arrays and records in arrays and records.

IMPORTANT: Unboxing is a **memory layout** property. You can not directly pass unboxed values to functions, nor return them. For example, signature `(#point) → i32` does not have a meaning in Barocq, but `([#[point, 2ul]]) → i32` does: the calling interface always expects pointers for non-primitive types. Thus, the access semantics for unboxed values is pointer-based. Expressions `x.f` and `y[i]` return a boxed value of type `t` if field `f` of `x` or elements of array `y` have layout `#t`. In this case, update operations `x.f ← v1` and `y[i] ← v2` are authorized only if `v1` (resp. `v2`) is a must-alias for the address of `x.f` (resp. `y[i]`): those updates actually become no-ops in the generated C code.

Expressions

Literals

Boolean literals are `true` and `false`.

Integer literals can be written in decimal (e.g. `123`) or hexadimal (e.g. `0xFF`) notation. By default, integer literals are parsed as being 32-bit signed integers. To change the signedness and size of a literal, the suffixes `u`/`U` (for unsigned), `l`/`L` (for long, i.e. 64-bit) and `ul`/`UL` (for unsigned 64-bit) must be used.

Record literals are written `{ f1 = l1, ⋯, fn = ln, }`, where `fi` are field names and `li` are the associated values that are also literals.

Array literals are written `[ l1, l2, ⋯, ln, ]` where each `li` is a literal.

IMPORTANT	Array and record literals can only be used for defining global constants and cannot appear in the body of a function.

Operators

Barocq comes with the usual unary and binary operators, which are the following.

Unary operators:

Operator	Description	Operand type
`!`	Boolean NOT	`bool`
`+`	Numerical PLUS	Any integer type
`-`	Numerical MINUS	Any integer type
`~`	Bitwise NOT	Any integer type

Binary operators:

The two operands of a binary operation must have the same type.

Operator	Description	Operands type
`&&`	Boolean AND	`bool`
`||`	Boolean OR	`bool`
`^^`	Boolean XOR	`bool`
`+`	Numerical ADD	Any integer type
`-`	Numerical SUBSTRACT	Any integer type
`*`	Numerical MULTIPLY	Any integer type
`/`	Numerical DIVIDE	Any integer type
`%`	Numerical MODULO	Any integer type
`<<`	Bitwise LEFT SHIFT	Any integer type
`>>`	Bitwise RIGHT SHIFT	Any integer type
`&`	Bitwise AND	Any integer type
`|`	Bitwise OR	Any integer type
`^`	Bitwise XOR	Any integer type
`<`	Numerical LESS THAN	Any integer type
`<=`	Numerical LESS or EQUAL THAN	Any integer type
`>`	Numerical GREATER THAN	Any integer type

Operator	Description	Operands type
`>=`	Numerical GREATER or EQUAL THAN	Any integer type
`==`	Equality test	Any primitive type
`!=`	Equality test negation	Any primitive type

Other expressions

The following table summarizes the other expressions of the language. All identifiers which begin by an `e` are themselves expressions.

Syntax	Description
`let x = e1 in e2`	Let binding
`if e1 then e2 else e3`	Conditional expression
`match e0 with p1 ⇒ e1 ⋯ pn ⇒ en end`	Pattern-matching*
`e.f`	Accessing a record field
`e1.f <- e2`	Modifying a record field
`e1[e2]`	Accessing an array element
`e1[e2] <- e2`	Modifying an array element
`e1 as ty`	Casting `e1` to an expression of type `ty`
`f(e1, ⋯, en)`	Calling function `f` with arguments `e1, ⋯, en`

*For pattern-matching, each pattern `pi` is either an enum constructor or the wildcard `_`.

Syntactic sugar

To update multiple fields of the same record "at the same time", the following syntax can be used:

```
{ p with  x <- 2, y <- 3, }
```

In this example, `p` should be a record variable, and `x` and `y` fields of the corresponding record type.

IMPORTANT

This syntax is desugarged into a sequence of single-field record updates that preserves the order in which the programmer updated the fields. For example, the code snippet above is desugared as `(p.x <- 2).y <- 3`. Using a different order may lead to different errors raised by the compiler if side effects do occur in the update (compiled) expressions.

Definitions

Global constants and functions can be defined with the keyword `defn`.

Constants

The syntax is:

```
defn const_name : const_type = const_expr
```

`const_name` is an identifier, `const_type` is a type and `const_expr` is a constant expression. A constant expression is either a literal, a constant variable or a boolean or an arithmetic expression.

The following example declare several constants:

```
defn max_size : u32 = 10U

defn nb_elem : u32 = max_size / 2U

defn x_coord_init : i32 = 6

defn my_array : array point = [{ x = x_coord_init, y = -2, },  { x = x_coord_init, y =
34, },]
```

IMPORTANT	Only constants with an integer or boolean type can be used in function bodies. Non-primitive constants may be used by manually written C code which would be linked to Barocq generated code.

Functions

The syntax is:

```
defn fun_name(p1: t1, ..., pn: tn) : tr =
  (* body *)
```

Each parameter `pi` is associated with its type `ti`. `tr` is the return type. A function can take no parameters.

NOTE	All functions are uncurried, partial application is not supported.

Foreign function interface

You may need to provide types and definitions that do not have an implementation in Barocq, but are implemented in C. Barocq supports abstract types and abstract definitions.

Abstract types

For abstract types, Barocq needs to now if the underlying type is a struct or a union:

```
type my_type1 of "struct"

type my_type2 of "union"
```

All references to an abstract type in the Barocq code are translated to a boxed type in the generated C file.

Abstract definitions

Abstract constants and functions are declared with the keyword `decl`:

```
type t1 of "struct"

record rec {
  f1: t1,
  f2: u32,
}

decl const1 : u32

decl const2 : rec

decl rand : (i32) -> i32

decl copy_array : (@read [point], @write [point]) -> [point]
```

By default, all arguments passed to an abstract function will become unsuable after the call (like in a strict linear type system.). To prevent this, you can use the annotation `@read` to specify that the function will only read data from the argument and will never modify it. This annotation is used in the abstract function `copy_array` for example.
The second existing annotation is `@write`, which means two things. First, it indicates that the function may modify the argument, and second, that the return value is a must-alias to this argument. Any abstract function that returns a non-primitive value must have a parameter annotated with `@write` with the same type as the return type.

IMPORTANT

The contract of abstract functions

Abstract functions must respect the following contract:

- They cannot make a newly allocated object escape from the local scope;
- They must preserve the memory layout of the call state (e.g. pointer moves are forbidden). If changes occur in the memory, thoses changes can only be a modification of a primitive value contained in the arguments;
- They cannot modify an argument annotated with `@read`;
- If the result is non-primitive, it must point to the argument annotated with `@write`.

Modules

Barocq supports multi-file compilation. Each file corresponds to a module, and must begin with the declaration of the module name:

```
module Mname
```

Module names are composed of letters, and the first one must be capitalized.

To reference an external idenfitier in the current module, the identifier must be prefixed with `Mname::` where `Mname` is the module name. The external identifiers can refer to a type, a constant, a function or an enum constructor.

It is also possible to import modules at the beginning of a file just after the module declaration:

```
module Mname

import Mod1
import Mod2
```

This example will bring the definitions of `Mod1` and `Mod2` in scope, so that using the module name prefix `Mod1::` or `Mod2::` is no longer mandatory. If `Mod1` and `Mod2` have definitions with the same name, the last imported module takes precedence (in the above example, it is `Mod2`).

NOTE	Use imports with care. Referencing an external definition without its module name can make the code harder to read and understand.

WARNING	When compiling with (dependent) multiple files, all the dependencies must be given in order to the compiler. The Barocq compiler does not support separate compilation (no linking phase) and does not have a dependency analyzer yet.

Compilation directives

Barocq supports the following compilation directives:

Directive	Associated with	Effects
`@inline`	A function definition	Produces the C `inline` attribute for the corresponding function
`@always_inline`	A function definition	Produces the C `always_inline` attribute code for the corresponding function
`@static`	A function definition	Produces the C `static` keyword for the corresponding function
`@all_static`	A whole module (must be placed after the imports)	Produces the C `static` keyword for all the module's functions
`@export`	A function definition	Removes the C `static` keyword for the corresponding function if the parent module was marked `[#static]`
`@unique`	A record-typed constant definition or declaration	Tells the compiler that the marked constant is the unique instance of its type

More details about `@unique`

In microkernel code, functions may operate on a (part of a) global data structure that represents the kernel state. In a functional programming language like Barocq, this global state must be passed as an argument to every function that reads from or writes to it. The `@unique` directive can be used to modify the generated code by specifying that a global variable will be the only instance of its type during a program execution. Let's say that we mark a global variable `g` of type `ty` with `@unique`, then this directive has the following effects:

- For function signatures, it removes every parameter of type `ty` and transforms the return type `ty` into `void`;
- In function bodies, if `x` and `y` are of type `ty`, then:
  - Every variable assignement `x = y` is removed;
  - Every record operations `x.f` and `x.f = v` are replaced by `g.f` and `g.f = v`;
  - Every function call assignment `x = foo(a1, ⋯, y, ⋯, an)` are transformed into `foo(a1, ⋯, an)` to match the new signature of `foo`;
  - Every statement `return x` is replaced by `return`.

To ensure that these transformations remain correct, marking a global variable with `@unique` also restricts the set of accepted programs. If `ty` is the type of this global variable, then the following things are **forbidden**:

- Record type definitions that contain a field of type `ty`;
- Functions for which `ty` appears more than once in the parameter types (e.g. function types `(t1, ty, ⋯, ty) → tr` or `([ty], t1, t2) → tr` are rejected).

A small example

A toy microkernel

In this example, we write of what could be an excerpt of a microkernel. We have two source files: `kernel.brc` and `machine.brc`.

First, let's look at `machine.brc`.

machine.brc

```
module Machine

type state of "struct"

decl write_timecmp : (@write state, u64) -> state
```

This module defines the FFI to interact with the underlying machine. It first declares an abstract type - the machine state - that will be implemented as a C struct. Then it declares function `write_timecmp`, that writes the time compare register of the machine state.

Now, let's look at `kernel.brc`.

kernel.brc

```
module Kernel

defn nb_procs : u64 = 5ul

defn quantum : u64 = 100ul

enum proc_status {
  READY,
  RUNNING,
}

record proc {
  pid: u64,
  regs: #[u64; 32ul],
  status: proc_status,
}

record state {
  curr_pid: u64,
  procs: #[#proc; nb_procs],
  deadline: u64,
  mc: Machine::state,
}
```

```
defn update_proc_status(ks: state, pid: u64, status: proc_status) : state =
  let procs = ks.procs in
  let proc = procs[pid] in
  let proc = proc.status <- status in
  let procs = procs[pid] <- proc in
  ks.procs <- procs

defn schedule(ks: state, now: u64) : state =
  if now > ks.deadline then
    let curr_pid = ks.curr_pid in
    let next_pid = (curr_pid + 1ul) % nb_procs in
    let next_deadline = now + quantum in
    let ks = update_proc_status(ks, curr_pid, READY) in
    let ks = update_proc_status(ks, next_pid, RUNNING) in
    let ks = { ks with curr_pid <- next_pid, deadline <- next_deadline, } in
    ks.mc <- Machine::write_timecmp(ks.mc, ks.deadline)
  else
    ks
```

The kernel state is composed of the current process id, the array of processes, the deadline of the current process and the machine state. A process is represented as a record containing its identifier, its set of saved registers and its status.

Two constants describe the kernel configuration: the number of processes `nb_procs`, and the quantum value for a round-robin scheduling `quantum`.

Finally, two core functions are defined. `update_proc_status` modifies the status of the given process. `schedule` chooses the next process to be scheduled. This function checks whether the current process has missed its deadline. If so, it selects the next process to run, updates the status of the current and next processes and increases the deadline.

You may notice that to modify the status of a process, `update_proc_status` contains all the steps from getting the process, modyfing its status and updating the process table back. In fact, Barocq does **not** support *deep updates* for the moment. If you try to write `ks.procs[next_id].status <- v`, this instruction is interpreted as:

1. < Retrieve the value from `ks.procs[next_id]` >, which returns a value of type `proc`;
2. < Update the field `status` of this value >, which also returns a value of type `proc`.

A deep update would make this instruction effectively return the new updated kernel `state`, but this is not yet supported by the compiler.

To compile those files with the Barocq compiler, we use the following command:

```
barocq -fcopy-prop -o kernel.c -gen-header -types-impl machine.h machine.brc
kernel.brc
```

This command will generate a C file `kernel.c` and the associated header file `kernel.h`. All definitions are grouped together in `kernel.c` and prefixed with the name of the module from which they

originate. For exemple, the function `schedule` becomes `Kernel_schedule`. The option `-types-impl` allows the user to specify the location of the file containing the implemention of all abstract types. This option appends the necessary `#include` directive at the beginning of `kernel.h` and `kernel.c`. You can find the source files and the compilation artefacts of this example under the `example` directory.

Generating Rocq files

The Barocq compiler can generate a shallow and a deep embedding of a Barocq program, as well as the correspondence proofs between them.

The following command generates all of the Rocq files (embeddings + correspondence proofs) in the `rocq` folder.

```
barocq -odir rocq -orocq kernel -gen-corres machine.brc kernel.brc
```

The correcntess of the translation to a Rocq shallow embedding is verified via an a-posteriori validation. This validation requires proving some correspondence theorems between the shallow embedding and the semantics given to the deep embedding. We divide the proofs by generating two shallow embeddings: a high-level one (`kernel_ShallowR.v`) that uses the native Rocq enums and records, and a low-level one (`kernel_ShallowB.v`) that uses the Barocq encoding of enums and records. The high-level embedding is well-suited for formal verification. The correspondence proofs are generated for ShallowR / ShallowB (`kernel_CorresRB.v`), ShallowB / Deep (`kernel_CorresBD.v`), and by transitivity, ShallowR / Deep (`kernel_CorresRD.v`).

Coding good practices

As mentionned in the preamble, all array and record modifications are compiled into in-place updates. The static analyzer of Barocq performs a points-to based analysis to check wether the source program can be compiled with in-place updates without breaking its semantics. We explain here the restrictions it induces and the good practices to follow to avoid getting errors from the static analyzer.

NOTE	The points-to analysis is performed on an intermediate representation called `Imp1`. The error messages are thus related to the `Imp1` syntax. Precisely pinpointing the origin of the problem in the source code is a tedious task that has not yet been done. To make the debugging of Barocq code easier, you can display the `Imp1` representation with the `-print imp1` option of the compiler.

The first thing to have in mind when you write Barocq code is that you are kind of writing an imperative program with a functional syntax. Thus, in the compiled code, assignments to non-primitive values actually create aliases, and modifying an array or a record updates the memory in-place. To make a functional program compatible with those behaviors, it is strongly recommended to adopt an *affine* programming style (a bit like Rust for those familiar with the language). This means that you should only use a non-primitive value at most once. More precisely, we adopt the following guideline: "everytime you make a modification to an object, never try to use the old version of it". This is what we do in the example above. To help following this good practice, we use variable shadowing every time we modify a variable, so that we are sure that we will only use the last reference to it.

The second thing to know concerns arrays. Usually, arrays are accessed with indices whose values are not statically known. Barocq enforces a very strict policy to make sure the programmer doesn't access an array of non-primitive elements with different indices at the same time. The restrictions on arrays of non-primitive elements are the following: (1) you can only write into one element at a time, (2) a function cannot return array element if the provided index does not have a meaning outside the function scope (e.g., you can return t[i + 1] if `i` is a function parameter, but not if it's a local variable).

Finally, you will not be able to move pointers with the actual implementation of Barocq. The modifications you can make in the memory can only correspond to updating primitive values in objects. This means that source-level record and array udpates of non-primitive values become no-ops in C as they just re-assign the same pointer.